

UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA,
ELECTRÓNICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Programacion Competitiva

tema: sudoku

Docente:

Ing. Aceituno Rojo Miguel Aceituno

Presentado por:

Pari Valencia Jhon Daniel

PUNO - PERÚ

2026

Guía de Solución de Sudoku mediante Backtracking

Este documento presenta el desarrollo completo para solucionar un Sudoku de 9x9 utilizando la técnica de Backtracking (Vuelta Atrás). Se incluye la explicación detallada del árbol de recursión y el código fuente optimizado en C++ basado en la plantilla provista.

1. Concepto de Backtracking en Sudoku

Al igual que en los árboles de recursión de Factorial o Fibonacci observados previamente, el Backtracking construye una solución paso a paso de forma jerárquica. Sin embargo, a diferencia de la recursión simple, el Backtracking incluye un proceso de poda estricta (evaluación de restricciones): si una decisión intermedia viola las reglas del Sudoku, el algoritmo descarta esa rama inmediatamente y regresa al estado anterior (retrocede) para probar una nueva opción.

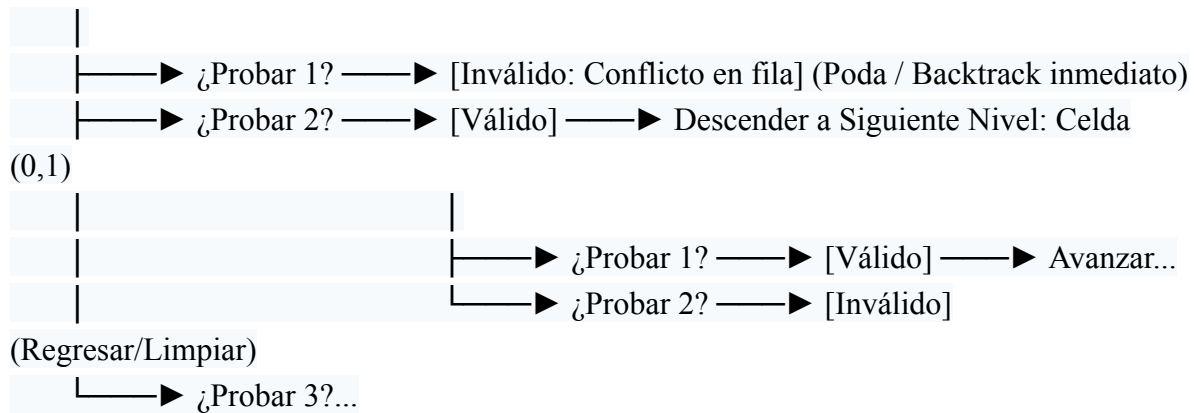
2. Representación del Árbol de Decisiones (Recursión)

En el problema del Sudoku, cada nodo del árbol representa una celda vacía específica (fila, columna). Las ramificaciones (hijos) representan los números del 1 al 9 que se intentan colocar de manera tentativa en esa posición.

Elemento del Árbol	Interpretación en Sudoku
Nodo Raíz	Estado inicial del tablero con los números fijos cargados desde el archivo de entrada.
Nivel del Árbol	Representa el procesamiento ordenado de cada una de las celdas vacías restantes.
Ramas (Hijos)	Opciones numéricas probadas (1 al 9). Si un número incumple las reglas básicas de no repetición en fila, columna o cuadrícula, la rama es podada (abortada).
Hojas Exitosas	Se alcanza cuando el algoritmo no encuentra más celdas vacías y valida la estructura final completa.

Esquema Visual de Flujo de la Rama

[Celda Vacía Detectada (0,0)]



3. Código Fuente Completo Corrección en C++

Se corrigieron y optimizaron las firmas de las funciones originales. Dado que Solucionar Sudoku necesita validar estados recursivos y retornar el éxito o fracaso de una rama completa, se definió como tipo bool. Además, se añadió la lógica de validación modular por cuadrantes.

```

#include <iostream>

#define MAX 9

using namespace std;

// verifica si es seguro colocar un valor en una celda específica
bool esSeguro(int tablero[9][9], int fila, int col, int num) {
    // Comprobar fila
    for (int x = 0; x < 9; x++) {
        if (tablero[fila][x] == num) return false;
    }

    // comprobar columna
    for (int x = 0; x < 9; x++) {
        if (tablero[x][col] == num) return false;
    }

    // comprobar subcuadrícula de 3x3
    int inicioFila = fila - fila % 3;

```

```

    int inicioCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (tablero[i + inicioFila][j + inicioCol] == num) return false;
        }
    }

    return true;
}

```

// función algorítmica principal utilizando Backtracking recursivo

```
bool SolucionarSudoku(int tablero[9][9]) {
```

```

    int fila = -1;
    int col = -1;
    bool vacio = true;

```

// encontrar la primera celda que contenga un 0 (celda libre)

```

    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (tablero[i][j] == 0) {
                fila = i;
                col = j;
                vacio = false;
                break;
            }
        }
        if (!vacio) break;
    }

```

// caso base: Si no quedan celdas vacías, el Sudoku está resuelto correctamente

```
if (vacio) return true;
```

// ciclo de ramificación para probar dígitos del 1 al 9

```

    for (int num = 1; num <= 9; num++) {
        if (esSeguro(tablero, fila, col, num)) {
            // asignación tentativa (descenso en el árbol de recursión)

```

```
    tablero[fila][col] = num;
```

```
    // evaluar de manera recursiva el subárbol resultante
```

```
    if (SolucionarSudoku(tablero)) return true;
```

```
    // si falla la asignación intermedia, se aplica Backtracking (limpieza de estado)
```

```
    tablero[fila][col] = 0;
```

```
    }
```

```
}
```

```
// retorna false si ninguna rama numérica de este nodo es viable
```

```
return false;
```

```
}
```

```
void Imprimir(int tablero[9][9]) {
```

```
    for (int fila = 0; fila < 9; fila++) {
```

```
        if (fila % 3 == 0 && fila != 0) {
```

```
            cout << "-----+-----+-----" << endl;
```

```
        }
```

```
        for (int col = 0; col < 9; col++) {
```

```
            if (col % 3 == 0 && col != 0) {
```

```
                cout << "| ";
```

```
            }
```

```
            cout << tablero[fila][col] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
int main() {
```

```
    // redirección de lectura desde archivo de entrada según la plantilla provista
```

```
    freopen("in.txt", "r", stdin);
```

```
    int tablero[9][9] = {};
```

```
    int pos_x, pos_y, valor;
```

```

// lectura dinámica indexando desde 0 internamente
while (cin >> pos_x >> pos_y >> valor) {
    tablero[pos_x - 1][pos_y - 1] = valor;
}

cout << "tablero inicial" << endl;
Imprimir(tablero);
cout << endl;

// ejecución y evaluación del resultado
if (SolucionarSudoku(tablero)) {
    cout << "sudoku solucionado" << endl;
    Imprimir(tablero);
} else {
    cout << "no se encontro una solucion valida para el tablero ingresado." << endl;
}

return 0;
}

```

4. Comparativa con Estructuras Previas

A diferencia de la recursión pura en *Factorial* (un único camino lineal) o *Fibonacci* (un árbol de decisión binario estático), el árbol del Sudoku posee un factor de ramificación variable de hasta 9 hijos por cada celda vacía. La inclusión de la validación activa representa la optimización mediante la eliminación inmediata de caminos inválidos, lo cual evita que la máquina explore miles de estados inviables.